UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación · Carrera de Software

Proyecto Fin de Curso, entrega final: sistema distribuido en capas, calidad y defensa

TA-PFC-E4 · Guía para el estudiante y rúbrica de calificación

Aplicaciones Distribuidas (ISR-701) · SPA 2026–2027

Campo	Detalle
Código	TA-PFC-E4
Componente	Trabajo Autónomo (TA)
Modalidad	Grupal, equipos del PFC de 3 o 4 integrantes, con defensa oral
Carácter	Sumativa
Semana	17 (15 al 21 de febrero de 2027), última semana del segundo corte
Cierre del repositorio	martes 16 de febrero de 2027, 23:59 (etiqueta de Git `e4`)
Defensas	Del miércoles 17 al viernes 19 de febrero de 2027, según el turno publicado en el SGA
Peso en la nota	1,00 de los 10 puntos del segundo corte
Formativa vinculada	FORM-TL-06 Revisión técnica del código del PFC (semana 15)
Actividades que integra	TA-PFC-E1 a E3, PE-U4 (GA-SUM-06) y PE-U5 (GA-SUM-07)
Docente	Gleiston C. Guerrero-Ulloa, PhD (`gguerrero@uteq.edu.ec`)

1. Propósito de la entrega

La entrega final cierra el PFC. El equipo presenta su sistema distribuido completo, organizado en capas, desplegable con un solo comando, probado en varios niveles, verificado automáticamente en cada cambio y observable mientras funciona; evalúa su calidad con el modelo ISO/IEC 25010:2023 [1]; y lo defiende oralmente. La entrega aplica, sobre el propio proyecto, el resultado de aprendizaje de la Unidad 5: *construye aplicaciones distribuidas en capas que cumplan estándares de calidad de software (ISO/IEC 25010), con seguridad y ética profesional, trabajando en equipo y en ciclos iterativos*, e integra lo construido en las entregas 1 a 3.

La idea central en una frase

Cualquier persona debe poder clonar el repositorio en la etiqueta `e4`, levantar el sistema completo con `docker compose up`, ver el *pipeline* en verde, abrir el panel de observabilidad y comprobar cada cifra del documento en un archivo de resultados. En la práctica, lo que no se puede reproducir desde el repositorio no se califica.

2. Qué debe construir y entregar el equipo

La entrega se organiza en ocho módulos. Cada uno indica entre corchetes el rol responsable sugerido; en equipos de tres integrantes, un estudiante asume dos roles. Todos los integrantes deben tener *commits* propios y sustantivos en la entrega.

Módulo A — Arquitectura en capas y patrones *[Arquitecto]*

1. Cada microservicio está organizado en capas (servicios web, negocio, dominio y acceso a datos) y cada capa usa solo a la inferior [2], [3].
2. Al menos **tres reglas de arquitectura automatizadas** (por ejemplo, con ArchUnit) que fallan si se viola la separación de capas, ejecutadas en el *pipeline*.
3. Al menos **tres patrones de diseño** aplicados y justificados (por ejemplo, Repository, Factory, Strategy, Observer o Proxy) [4], [5], con la clase donde se aplican y el problema que resuelven.
4. Principios SOLID verificados con evidencia [6] y diagramas C4 actualizados en los niveles 1, 2 y 3.
5. Las decisiones de arquitectura (ADR) de las entregas anteriores, actualizadas, más al menos una nueva sobre la arquitectura en capas.
6. **Todos los *issues* de FORM-TL-06** y de la revisión docente cerrados por *pull requests* revisados, o justificados en el documento si se decidió no resolverlos.

Módulo B — Sistema integrado y desplegable *[Arquitecto]*

1. Un único `docker compose up` levanta, desde un clon limpio, todos los componentes: API Gateway, microservicios, base de datos distribuida de la entrega 3, *broker*, componente de procesamiento, cliente web o móvil y herramientas de observabilidad.
2. Verificaciones de salud en todos los servicios, arranque condicionado a la salud de sus dependencias, volúmenes persistentes y configuración por variables de entorno con `.env.example`. Ningún secreto real en el repositorio.
3. Imágenes con construcción en varias etapas y ejecución con un usuario sin privilegios.

Módulo C — Pruebas en varios niveles *[Responsable de calidad]*

Ninguna técnica de prueba cubre por sí sola un sistema de servicios distribuidos; se necesitan varios niveles complementarios [7], [8].

1. **Unitarias** de la capa de negocio y del dominio, con cobertura de líneas de al menos el 70 % en esas capas, medida con una herramienta (por ejemplo, JaCoCo o coverage.py) y con el informe versionado.
2. **De integración** de la capa de acceso a datos contra la base de datos real en un contenedor (por ejemplo, Testcontainers).
3. **De la API**: cada operación del contrato OpenAPI responde con los códigos y los esquemas documentados, incluidos los errores.
4. **De extremo a extremo**: al menos un flujo completo del negocio a través del API Gateway.
5. **De concurrencia**: la prueba de la entrega 3 sobre el recurso compartido, ahora automatizada.
6. **De carga**: dos escenarios (50 usuarios simultáneos durante 5 minutos, y una rampa de 0 a 200 usuarios en 10 minutos) con una herramienta de carga, repetidos al menos tres veces, reportando rendimiento (solicitudes por segundo), latencia en los percentiles 50, 95 y 99 y tasa de errores, con su dispersión [9].

Módulo D — Integración y entrega continuas *[Responsable de calidad]*

1. Un flujo de GitHub Actions que, en cada *push* y *pull request*, ejecuta análisis estático, pruebas unitarias, reglas de arquitectura, pruebas de integración y construcción de imágenes, en trabajos separados.
2. La publicación de imágenes en GitHub Container Registry, etiquetadas con el identificador del *commit*, depende de que todos los trabajos anteriores terminen en verde.
3. El *pipeline* está en verde en el *commit* de la etiqueta `e4`, y la rama principal está protegida para no aceptar cambios con el *pipeline* en rojo. La automatización de la construcción, las pruebas y el despliegue es central para cambiar con frecuencia un sistema distribuido [10].

Módulo E — Observabilidad *[Arquitecto]*

1. Registros estructurados en formato JSON en todos los servicios, con un identificador de traza que se propaga entre servicios, siguiendo el estándar W3C Trace Context [11].
2. Métricas expuestas por cada servicio y recolectadas con Prometheus, con al menos dos métricas de negocio propias del dominio del PFC.
3. Un panel de Grafana con al menos seis vistas: tasa de solicitudes, latencia en los percentiles 50, 95 y 99, tasa de errores de servidor, estado de los nodos de la base de datos distribuida, uso de recursos por contenedor y una métrica de negocio. El panel se versiona como JSON.
4. Errores de la API en formato Problem Details (RFC 9457) [12]. Sin observabilidad, un sistema distribuido en producción no se puede diagnosticar [13].

Módulo F — Evaluación de calidad con ISO/IEC 25010:2023 *[Responsable de calidad]*

Construya una matriz con al menos **cuatro características** del modelo de calidad del producto priorizadas para el dominio del PFC. Para cada una: por qué se prioriza, la métrica con la que se mide, la meta, el valor medido y la evidencia en el repositorio (prueba, medición, configuración o captura). Una característica sin valor medido no se cuenta.

Módulo G — Documento LaTeX final *[todo el equipo]*

El documento es acumulativo: integra y actualiza las entregas 1 a 3 y añade los capítulos de esta entrega. Contiene, como mínimo:

a) resumen del sistema y de su evolución desde la entrega 1;

b) arquitectura final: diagramas C4, capas, patrones, SOLID, ADR y reglas automatizadas (módulo A);

c) despliegue (módulo B) y *pipeline* (módulo D), con capturas del *pipeline* en verde;

d) pruebas: estrategia, cobertura medida y resultados de carga con tablas y gráficos (módulo C);

e) observabilidad: instrumentación y capturas del panel durante la prueba de carga (módulo E);

f) matriz de calidad ISO/IEC 25010:2023 (módulo F);

g) resolución de los *issues* de FORM-TL-06;

h) limitaciones, riesgos abiertos y trabajo futuro;

i) tabla de aportes de cada integrante en las cuatro entregas, con sus *commits* principales;

j) declaración de uso de inteligencia artificial y referencias en formato IEEE (al menos diez, seis de ellas indexadas).

Módulo H — Defensa oral *[todo el equipo]*

La defensa dura 20 minutos por equipo: 15 de exposición con demostración en vivo y 5 de preguntas. Cada integrante expone al menos tres minutos y todos responden preguntas. Sigue las cinco fases de las exposiciones de la asignatura:

Fase	Tiempo	Contenido
F1	1–2 min	El problema del dominio, el dato más contundente del sistema y los roles del equipo.
F2	3 min	Arquitectura final en capas, con un diagrama propio y las decisiones principales.
F3	5 min	Demostración en vivo desde el clon de la etiqueta `e4`: el sistema levantado, un flujo completo del negocio con su traza en el panel, y la caída de un servicio o de un nodo de la base de datos con su efecto observado.
F4	3 min	Resultados de las pruebas de carga y matriz de calidad.
F5	2–3 min + 5 de preguntas	Tres conclusiones propias, limitaciones y defensa técnica.

3. Estructura mínima del repositorio y forma de entrega

```
/                     README.md, compose.yaml, .env.example, .gitignore
/.github/workflows/   flujo de integracion y entrega continuas
/services/<servicio>/ codigo, pruebas y Dockerfile de cada servicio
/gateway/             configuracion del API Gateway
/processing/          componente de procesamiento (entrega 3 y PE-U4)
/client/              cliente web o movil
/ops/                 prometheus/, grafana/pfc-dashboard.json, registros/
/tests/load/          scripts de carga y resultados crudos (CSV)
/docs/                .tex, .bib, figuras, PDF, adr/ y defensa/
```

1. El `README.md` explica cómo levantar el sistema, ejecutar cada tipo de prueba, reproducir las pruebas de carga, abrir el panel y compilar el documento (compilador, archivo principal, orden de comandos y dependencias).
2. Las diapositivas de la defensa se elaboran en LaTeX con `beamer` en `docs/defensa/`.
3. La entrega se marca con la etiqueta `e4`, creada y enviada hasta el martes 16 de febrero de 2027, 23:59:

```
git tag -a e4 -m "PFC entrega final"
git push origin e4
```

4. En la tarea del SGA, cada integrante sube **solamente un documento PDF de UNA SOLA página** con los datos identificativos (universidad, facultad, carrera, asignatura, código y nombre de la actividad, integrantes, paralelo y fecha) y la URL del repositorio escrita completa en una

sola línea.

4. Evidencias de autoría

1. **Historial continuo** a lo largo de las semanas 15 a 17, con *commits* propios de cada integrante y mensajes descriptivos. Una carga masiva al final es un indicio de autoría no propia.
2. **Resultados que se regeneran**: cada cifra del documento está en un archivo de resultados y se regenera con su *script*. Las cifras escritas a mano o producidas por un programa que las inventa se consideran fabricadas.
3. ***Pipeline* verificable**: las ejecuciones de GitHub Actions del *commit* etiquetado están disponibles para el docente.
4. **Defensa**: cualquier integrante debe poder explicar y modificar en el momento cualquier parte del sistema.

5. Criterios de piso

Su incumplimiento implica calificación CERO en la entrega

P1. PDF de una sola página con carátula y URL. Cada estudiante sube al SGA solamente un documento PDF de UNA SOLA página con los datos identificativos y, en una sola línea, la URL del repositorio donde está todo el trabajo. Si el PDF tiene más de una página, si se suben otros archivos en lugar de este PDF, si la URL no cabe en una sola línea o está rota, o si el repositorio no es accesible (privado o inexistente), la calificación es CERO.

P2. PDF reproducible. Si el PDF del documento no se genera correctamente al clonar el repositorio en la etiqueta `e4` y compilar el archivo `.tex`, la calificación es CERO. Las instrucciones de compilación (compilador, orden de comandos, archivo principal y dependencias) deben estar en el `README.md`.

P3. Plazo. La etiqueta `e4` no existe o apunta a contenido subido después del martes 16 de febrero de 2027, 23:59.

P4. Integridad. Plagio, contenido generado por IA generativa presentado como propio, resultados, capturas o demostraciones fabricados, o **una sola** referencia inexistente o cuyo DOI lleve a una obra con datos diferentes a los citados [14].

P5. Formativa vinculada. El estudiante que no entregó la formativa FORM-TL-06 no puede presentar esta entrega y obtiene CERO en ella, salvo justificación aprobada.

P6. Defensa. La inasistencia no justificada a la defensa implica CERO para el estudiante que falta; el resto del equipo defiende y se califica con normalidad.

6. Rúbrica del equipo

Los pesos de las dimensiones son los del encuadre para las entregas del PFC (D1 15, D2 15, D3 30, D4 20, D5 10, D6 10).

Tabla 1: Indicadores de la nota del equipo

Ind.	Indicador	Módulo y evidencia	Peso
D1 Cumplimiento y completitud (15)			
1.1	Entregables y reproducibilidad	Repositorio, README, etiqueta `e4`, sistema completo desde un clon limpio	10
1.2	Cierre de los *issues* de FORM-TL-06	*Issues* cerrados por *pull requests* revisados	5
D2 Fundamentación conceptual (15)			
2.1	Arquitectura en capas, patrones, SOLID y ADR	Módulo A y capítulo b) del documento	15
D3 Desarrollo técnico y evidencia (30)			
3.1	Sistema integrado y desplegable	Módulo B	6
3.2	Pruebas en varios niveles	Módulo C: unitarias con cobertura, integración, API, extremo a extremo, concurrencia	8
3.3	Integración y entrega continuas	Módulo D	6
3.4	Observabilidad	Módulo E	6
3.5	Reglas de arquitectura automatizadas	Módulo A, en el *pipeline*	4
D4 Análisis crítico y resultados (20)			
4.1	Pruebas de carga y su análisis	Módulo C: dos escenarios, repeticiones, percentiles, panel durante la carga	8
4.2	Evaluación ISO/IEC 25010:2023	Módulo F	7
4.3	Limitaciones y trabajo futuro	Capítulo h) del documento	5
D5 Comunicación (10)			
5.1	Documento LaTeX	Módulo G	4
5.2	Defensa oral del equipo	Módulo H: fases, demostración en vivo, tiempo	6
D6 Referencias, ética e integridad (10)			
6.1	Referencias, aportes y declaración de IA	Al menos diez fuentes IEEE, seis indexadas; tabla de aportes; declaración de IA	10
	Total		**100**

Tabla 2: Rúbrica del equipo: descriptores de nivel por indicador

Indicador (peso)	4 Excelente	3 Bueno	2 Aceptable	1 Insuficiente	0 Ausente
1.1 Entregables (10)	Desde un clon limpio en `e4`, siguiendo el README, se levanta el sistema completo, se ejecutan todas las pruebas y se compila el documento.	Un paso requiere un ajuste menor.	Un componente no se levanta o una prueba no se reproduce.	El sistema se levanta solo en parte.	No hay entrega.
1.2 Issues de FORM-TL-06 (5)	Todos cerrados por *pull requests* revisados por otro integrante, o justificados.	Uno o dos abiertos sin justificar.	La mitad abiertos.	Casi todos abiertos.	Sin *issues*.
2.1 Arquitectura (15)	Capas respetadas en todos los servicios; tres patrones o más con el problema que resuelven; SOLID con evidencia; C4 en tres niveles; ADR actualizados.	Correcto con una violación menor de capas o un patrón mal justificado.	Capas mezcladas en algún servicio o patrones solo declarados.	Arquitectura que no corresponde al código.	Sin arquitectura documentada.
3.1 Integración (6)	Un `docker compose up` levanta todo con salud, dependencias, volúmenes, `.env.example`, imágenes en varias etapas sin privilegios y sin secretos.	Falta un elemento menor.	Un componente fuera del *compose* o sin verificación de salud.	Requiere pasos manuales.	No se levanta.
3.2 Pruebas (8)	Cinco niveles presentes y en verde; cobertura medida de al menos 70 % en negocio y dominio con informe versionado.	Un nivel débil o cobertura entre 60 y 69 %.	Faltan dos niveles o cobertura sin medir.	Solo pruebas unitarias.	Sin pruebas.
3.3 CI/CD (6)	Trabajos separados, publicación en GHCR condicionada a que todo esté en verde, rama protegida, *pipeline* en verde en `e4`.	Falta una condición.	Publica sin depender de las pruebas o *pipeline* en rojo en `e4`.	*Pipeline* que no ejecuta pruebas.	Sin *pipeline*.
3.4 Observabilidad (6)	Registros JSON con identificador de traza propagado entre servicios, métricas con dos de negocio, panel de seis vistas versionado, errores RFC 9457.	Falta un elemento.	Sin propagación de la traza o panel incompleto.	Solo registros de texto.	Sin observabilidad.
3.5 Reglas automatizadas (4)	Tres reglas o más de capas que se ejecutan en el *pipeline* y fallan ante una violación.	Dos reglas.	Una regla, o reglas fuera del *pipeline*.	Reglas que no se ejecutan.	Sin reglas.

Indicador (peso)	4 Excelente	3 Bueno	2 Aceptable	1 Insuficiente	0 Ausente
4.1 Carga (8)	Dos escenarios con tres repeticiones o más, rendimiento, percentiles 50, 95 y 99, errores con dispersión, capturas del panel durante la carga e interpretación de los cuellos de botella.	Completo con interpretación superficial.	Un escenario o una sola corrida.	Cifras sin datos crudos.	Sin pruebas de carga.
4.2 ISO/IEC 25010 (7)	Cuatro características o más priorizadas para el dominio, con métrica, meta, valor medido y evidencia en el repositorio.	Una característica sin evidencia sólida.	Características sin valor medido.	Matriz genérica.	Sin evaluación.
4.3 Limitaciones (5)	Limitaciones y riesgos concretos derivados de los resultados, con trabajo futuro priorizado.	Correcto pero poco priorizado.	Limitaciones genéricas.	Una mención superficial.	Ausente.
5.1 Documento (4)	Acumulativo y actualizado, en el orden del módulo G, sin desbordes ni referencias sin resolver.	Defectos menores.	Capítulos de entregas anteriores sin actualizar.	Documento desordenado.	Sin documento legible.
5.2 Defensa del equipo (6)	Cinco fases en tiempo, demostración en vivo desde la etiqueta con los tres elementos exigidos, todos exponen al menos tres minutos.	Un elemento de la demostración falla o se excede el tiempo.	Demostración parcial o algún integrante expone poco.	Demostración no funciona.	No defiende.
6.1 Referencias e integridad (10)	Diez referencias IEEE o más, seis indexadas, verificables y citadas donde se usan; tabla de aportes de las cuatro entregas; declaración de IA.	Algún campo omitido.	Menos de diez o menos de seis indexadas.	Mal citadas o sin declaración de IA.	Sin referencias o en formato distinto de IEEE.

Nota sobre 6.1: una referencia inexistente, o cuyo DOI lleve a una obra con datos diferentes a los citados, no se califica aquí: implica calificación CERO en la entrega (criterio de piso P4).

7. Rúbrica del aporte individual

El aporte individual se califica con el historial del repositorio de todo el PFC hasta la etiqueta `e4` (unificando las identidades de Git de cada estudiante), la tabla de aportes y la defensa.

Tabla 3: Criterios del aporte individual

Criterio	Nivel 4 (Excelente)	Peso
A1 Amplitud e importancia	Sus *commits* cubren al menos un módulo completo de esta entrega y son necesarios para que el sistema funcione o los resultados existan.	25
A2 Complejidad técnica	Su aporte incluye trabajo técnico no trivial (servicios, pruebas, *pipeline*, observabilidad, medición), no solo formato.	20
A3 Autoría en el documento	Redactó capítulos que corresponden a lo que construyó, verificables en el historial del `.tex`.	15
A4 Continuidad	*Commits* distribuidos en las semanas 15 a 17 y en las entregas anteriores, sin cargas masivas al final.	15
A5 Defensa individual	Expone al menos tres minutos sin leer y responde con acierto las preguntas que se le dirigen, incluidas las de partes que no construyó.	25
Total		**100**

Los niveles 3, 2, 1 y 0 siguen la escala general del encuadre. Un estudiante sin *commits* propios obtiene 0 en A1 a A4.

8. Cómo se calcula la nota

$$\text{Nota individual} = 0{,}60 \times \text{Nota del equipo} + 0{,}40 \times \text{Nota del aporte individual}$$

Cada indicador se califica con un nivel de 0 a 4 según su calidad, no por su sola presencia; un indicador ausente, no verificable o falso vale 0 [15].

$$\text{Nota previa} = \sum_i \frac{w_i \times n_i}{4} \times \frac{10}{100}, \qquad \text{Nota final} = \max(0,\ \text{Nota previa} - \text{Penalizaciones})$$

En la retroalimentación se informan siempre la nota previa y la nota final. La nota aporta al segundo corte con su peso: Nota/10 × 1,00.

9. Penalizaciones que no anulan la nota

Situación detectada	Efecto (sobre 10)
Falta la declaración de uso de inteligencia artificial	−0,3
Referencias en formato APA u otro distinto de IEEE	6.1 = 0
Dos indicios de uso de IA generativa o de autoría no propia, I1 e I3 a I8 del encuadre (nivel B)	−1,5
Tres o más indicios, o resultados no verificados presentados como verificados (nivel C)	−3,0

10. Lista de verificación antes de crear la etiqueta

Revise la entrega con esta lista

- ☐ Desde un clon limpio, `docker compose up` levanta todos los componentes y todos responden a su verificación de salud.
- ☐ El *pipeline* está en verde en el *commit* que se va a etiquetar.
- ☐ Las reglas de arquitectura fallan si se viola una capa (pruébelo una vez a propósito).
- ☐ La cobertura de negocio y dominio es de al menos 70 % y su informe está versionado.
- ☐ Las pruebas de carga tienen tres repeticiones por escenario y datos crudos en CSV.
- ☐ Una solicitud deja el mismo identificador de traza en los registros de todos los servicios que atraviesa.
- ☐ El panel de Grafana tiene seis vistas y está versionado.
- ☐ La matriz ISO/IEC 25010 tiene valor medido y evidencia en cada característica.
- ☐ Todos los *issues* de FORM-TL-06 están cerrados o justificados.
- ☐ El documento se compila desde un clon limpio, tiene la tabla de aportes y la declaración de IA, y al menos diez referencias verificadas.
- ☐ Las diapositivas de la defensa están en `docs/defensa/` y el equipo ensayó la demostración desde un clon limpio.

11. Retroalimentación y revisión

El docente publica en el SGA la calificación con la retroalimentación individual de cada integrante, que incluye la nota del equipo, la nota del aporte individual, la nota previa y la nota final. Ante un desacuerdo, el estudiante lo plantea primero al docente, que responde en tres días; si persiste, acude a la Coordinación de Carrera.

Referencias

[1] International Organization for Standardization, *ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO, Geneva, Switzerland, 2023.

[2] M. Richards y N. Ford, *Fundamentals of Software Architecture, A Modern Engineering Approach*, 2.ª ed. O'Reilly Media, 2025, ISBN: 978-1-098-17551-1.

[3] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.ª ed. Addison-Wesley, 2021, ISBN: 978-0-13-688609-9.

[4] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns, Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, ISBN: 978-0-201-63361-0.

[5] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, ISBN: 978-0-321-12742-6.

[6] R. C. Martin, *Clean Architecture, A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 978-0-13-449416-6.

[7] F. Ponce, R. Verdecchia, B. Miranda y J. Soldani, «Microservices Testing: A Systematic Literature Review,» *Information and Software Technology*, vol. 188, pág. 107 870, 2025. DOI: 10.1016/j.infsof.2025.107870.

[8] A. Golmohammadi, M. Zhang y A. Arcuri, «Testing RESTful APIs: A Survey,» *ACM Transactions on Software Engineering and Methodology*, vol. 33, n.º 1, págs. 1-41, 2023. DOI: 10.1145/3617175.

[9] T. Hoefler y R. Belli, «Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results,» en *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC15)*, Austin, TX, USA: ACM, 2015, págs. 1-12. DOI: 10.1145/2807591.2807644.

[10] M. Waseem, P. Liang y M. Shahin, «A Systematic Mapping Study on Microservices Architecture in DevOps,» *Journal of Systems and Software*, vol. 170, pág. 110 798, 2020. DOI: 10.1016/j.jss.2020.110798.

[11] S. Kanzhelev, M. McLean, A. Reitbauer, B. Drutu, N. Molnar e Y. Shkuro. «Trace Context.» W3C Recommendation, World Wide Web Consortium. (23 de nov. de 2021), dirección: https://www.w3.org/TR/2021/REC-trace-context-1-20211123/ (visitado 26-10-2026).

[12] M. Nottingham, E. Wilde y S. Dalal, «Problem Details for HTTP APIs,» RFC Editor, RFC 9457, jul. de 2023. DOI: 10.17487/RFC9457. dirección: https://www.rfc-editor.org/rfc/rfc9457.

[13] S. Niedermaier, F. Koetter, A. Freymann y S. Wagner, «On Observability and Monitoring of Distributed Systems – An Industry Interview Study,» en *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings*, S. Yangui, I. Bouassida Rodriguez, K. Drira y Z. Tari, eds., ép. Lecture Notes in Computer Science, vol. 11895, Cham: Springer, 2019, págs. 36-52. DOI: 10.1007/978-3-030-33702-5_3.

[14] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. dirección: https://www.acm.org/code-of-ethics (visitado 26-10-2026).

[15] E. Panadero y A. Jonsson, «A critical review of the arguments against the use of rubrics,» *Educational Research Review*, vol. 30, pág. 100 329, 2020. DOI: 10.1016/j.edurev.2020.100329.